TRUF: DESIGN AND DEVELOPMENT OF AN INTEGRATED TURF BOOKING AND SPORTS COMMUNITY PLATFORM WITH SUBSTITUTE PLAYER RECRUITMENT

S. Evandle Benni **Praveen Keshavan**
Department of Computer Science and Design
Rajalakshmi Engineering College, Chennai, Tamil Nadu, India

***Abstract*—** The rapid urbanization of recreational sports infrastructure has generated significant operational challenges in venue slot reservation, player roster management, and financial accounting. Traditional booking approaches rely heavily on phone calls, unorganized instant messaging threads, and manual paper registries. These informal practices frequently result in scheduling double-bookings, unverified payment receipts, and high match cancellation rates stemming from unexpected player dropouts. This paper presents TRUF, an integrated, full-stack web-based application designed to consolidate sports facility management, match discovery, and substitute player recruitment into a unified ecosystem. Developed using Next.js, React, TypeScript, Tailwind CSS, PostgreSQL, and Prisma ORM, TRUF provides role-tailored dashboards for players, venue operators, and platform administrators. Authentication and request authorization are governed by Better Auth and Zod server-side validation. Financial transactions are integrated using the Razorpay API gateway operating in Test Mode, complemented by transactional email notifications powered by Resend. A central technical contribution of TRUF is its state-driven substitute player recruitment mechanism, which automatically flags roster vacancies caused by participant withdrawals and exposes open slots to the player community. This paper presents the system architecture, entity-relationship models, API transaction flows, and operational evaluation of the deployed platform (https://truf-red.vercel.app).

***Keywords*—** Turf Booking Engine, Sports Community Platform, Substitute Player Recruitment, Full-Stack Architecture, Next.js, Prisma ORM, PostgreSQL.

I. INTRODUCTION

Commercial synthetic sports turfs catering to amateur football, box cricket, and badminton have expanded rapidly across metropolitan regions in India. However, facility management software adoption among small and medium sports venue operators remains fragmented [1]. Operators typically record reservations manually or communicate with customers via informal messaging applications. This lack of centralized state management leads to availability conflicts, uncoordinated partial payments, and lost revenue.

From the athlete perspective, organizing a recreational game requires substantial manual effort. Host players must reserve a slot, coordinate participant schedules, divide venue rental fees, and confirm team rosters. A prominent operational failure in amateur sports organization is late player withdrawal. When a confirmed participant drops out shortly before a scheduled match, the remaining players must either absorb additional costs or cancel the session entirely due to insufficient headcount.

II. RESEARCH BACKGROUND

Modern web frameworks have evolved to support server-side rendering, type-safe API communication, and dynamic database ORMs [2]. While enterprise resource planning (ERP) platforms exist for formal sports clubs, amateur recreational venues require lightweight, low-latency, consumer-facing web architectures that integrate both booking logistics and social participant coordination.

III. PROBLEM FORMULATION

The operational shortcomings of traditional sports facility reservation and match organization can be formalized into three primary domains:

1) Slot Contention & Administrative Overhead: Absence of real-time slot locking leads to concurrency collisions during peak hours, increasing administrative overhead for turf owners.

2) Fragile Roster Dependencies: Conventional booking engines treat a booking as a static transaction between one customer and the venue. They offer no native mechanisms to handle participant dropouts or fill roster vacancies dynamically.

3) Opaque Financial Reconciliation: Manual payment collections via standalone UPI applications lack automated linkage to specific booking slots, complicating refund workflows and revenue analytics.

IV. RESEARCH OBJECTIVES

To address these challenges, the TRUF project was undertaken with the following explicit technical goals:

1) Design a multi-role web platform supporting Players, Venue Owners, and System Administrators with strict permission boundaries.

2) Construct an ACID-compliant database architecture using PostgreSQL and Prisma ORM to prevent double-booking anomalies during peak reservation periods.

3) Implement a state-driven substitute player recruitment engine that detects team vacancies and publishes recruitment listings automatically.

4) Integrate payment verification simulation using Razorpay Test Mode and transaction notification dispatches via Resend API.

V. RELATED WORK AND LITERATURE REVIEW

Online reservation systems have been extensively studied across hospitality, healthcare, and venue management domains. Albrecht et al. [2] highlighted that centralized real-time synchronization reduces administrative booking errors by over 80% compared to asynchronous manual booking processes.

In sports technology, Kumar and Sharma [3] examined venue discovery platforms, demonstrating that transparent pricing and dynamic slot calendars significantly improve facility utilization. However, their scope was restricted to two-party business-to-consumer (B2C) booking interactions.

Social match coordination was analyzed by Chen and Park [4], who established that community discovery features increase amateur sports participation rates. Furthermore, Tan et al. [5] demonstrated that open-roster participant substitution workflows substantially lower event cancellation probability in recreational leagues.

Commercial applications such as Playo [6] and Hudle [7] have commercialized venue reservations in major metropolitan markets. Nevertheless, existing platforms frequently separate court booking from player substitution workflows, requiring users to coordinate player replacements across external messaging applications. TRUF bridges this gap by integrating substitute recruitment directly into the booking entity lifecycle. Table I presents a structured comparison of these approaches.

TABLE I
COMPARATIVE ANALYSIS OF RESERVATION AND MATCHMAKING SYSTEMS

System Dimension	Manual Phone/Chat	Generic Booking Engines	TRUF Platform
Slot Synchronization	Manual Register / Error-Prone	Real-Time Database Lock	ACID Relational Sync (Prisma/Postgres)
Payment Verification	Unverified Manual UPI	Production Payment Gateway	Razorpay Test Flow & HMAC Signature Audit
Substitute Recruitment	External Chat Groups	Static Bulletin Board	State-Driven Recruitment Workflow Engine
Role Management	None	Player & Owner Roles	Triple Role (Player, Owner, Administrator)
Owner Analytics	Manual Notebook / Excel	Basic Sales Figures	Comprehensive Trend & Refund Analytics

VI. PROPOSED SYSTEM AND METHODOLOGY

The TRUF platform is designed around a component-driven software engineering methodology. The platform architecture was developed following rigorous user role analysis:

1) Player Module: Enables users to discover venues, view real-time slot availability, reserve playing areas, organize public/private matches, initiate substitute recruitment upon dropout, and view match history.

2) Turf Owner Module: Allows facility managers to register venue profiles, define operating hours, set hourly pricing tiers, monitor slot occupancy, review financial statements, and manage refund processing.

3) Administrator Module: Empowers platform administrators to audit venue listings, enforce security policies, review system audit logs, and manage user authorizations.

Fig. 1 illustrates the operational flow of the proposed system methodology, highlighting the lifecycle from venue setup to slot reservation and substitute recruitment fulfillment.

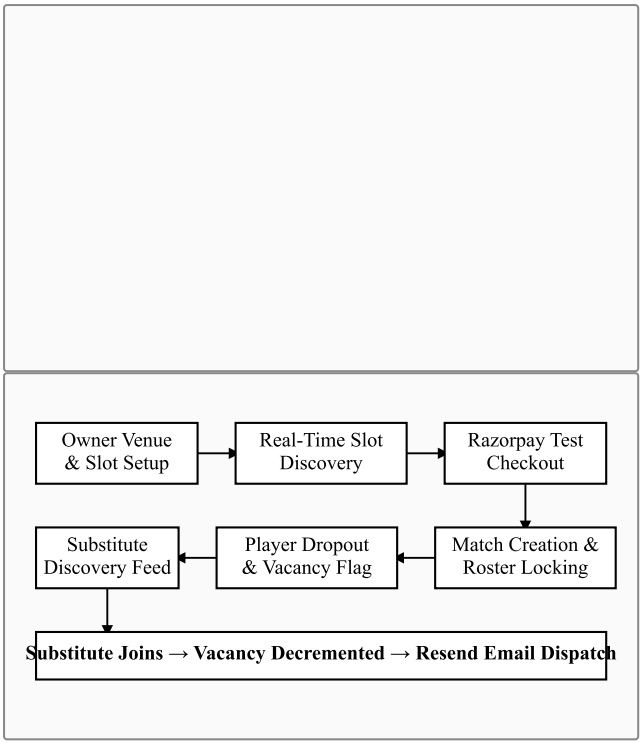


Fig. 1. System operational flow diagram outlining venue initialization, user booking execution, roster locking, and substitute recruitment workflow.

VII. SYSTEM ARCHITECTURE

TRUF uses a modern multi-tiered full-stack web architecture leveraging Next.js App Router capabilities. Presentation, business logic, and data persistence layers are cleanly decoupled to maintain maintainability and computational efficiency. Fig. 2 details the interaction layers.

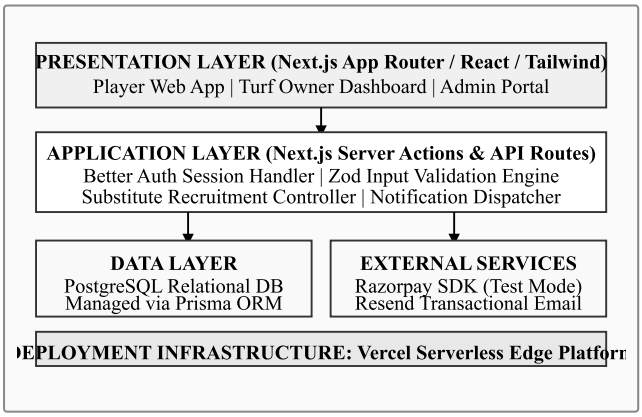


Fig. 2. Multi-tier system architecture diagram showing structural interactions between client components, server actions, database abstractions, and external service APIs.

1) Frontend & Presentation Tier: Built using TypeScript, React 18, and Next.js 14. Modern utility-first Tailwind CSS is used for responsive UI component styling across desktop and mobile devices.

2) Application & Server Tier: Encapsulates server actions and API route handlers. Session authentication is governed by *Better Auth*, enforcing HTTP-only cookie management and token validation. Runtime payload validation is enforced using *Zod* schemas [8].

3) Data & External Integration Tier: Utilizes cloud PostgreSQL managed via *Prisma ORM* [9]. Financial payment flows use Razorpay SDK (Test Mode), while email dispatches are processed asynchronously via Resend API [10]. Hosting is provided by Vercel [11].

VIII. SUBSTITUTE PLAYER RECRUITMENT MECHANISM

The core functional innovation of TRUF is its deterministic substitute player recruitment engine. Late player dropouts represent a severe operational pain point in amateur sports. TRUF resolves this through a structured roster state machine.

When a participant cancels their registration before the match lockdown threshold, the server executes an atomic database transaction. The participant state transitions to `WITHDRAWN`, and the parent `Match` record increments its `vacantSlots` property while setting its status flag to `RECRUITING`.

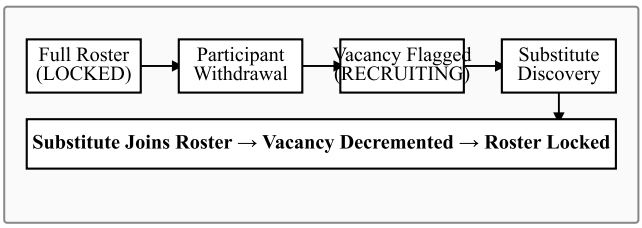


Fig. 3. State-transition workflow diagram of the substitute player recruitment module.

Vacant matches are immediately indexed in the public *Substitute Match Feed*. Community players can discover open positions filtered by sport, geography, skill level, and schedule. When a substitute registers, the roster state is decremented, restoring the match to `FILLED` status upon complete roster re-fulfillment. This programmatic recruitment workflow eliminates manual group messaging and preserves match scheduling viability.

IX. TECHNOLOGY STACK

The software stack was selected to achieve optimal runtime performance, developer efficiency, and structural type safety:

- **Next.js & React:** Provides server-rendered pages, API routes, optimized asset delivery, and responsive component UI hydrations.
- **TypeScript & Tailwind CSS:** Enforces compile-time type checking and utility-first responsive layout styling.
- **PostgreSQL & Prisma ORM:** Delivers robust relational data storage, transactional integrity, and programmatic database migrations.
- **Better Auth & Zod:** Secures user authentication sessions and validates HTTP payload schema contracts.
- **Razorpay (Test Mode) & Resend:** Handles simulated digital payment checkouts and dispatches transactional confirmation emails.

X. DATABASE DESIGN

The PostgreSQL database structure consists of 16 core relational tables configured with strict foreign-key constraints. Key persisted entities include `User`, `Session`, `Venue`, `PlayingArea`, `Slot`, `Booking`, `Match`, `Participant`, `PaymentOrder`, `Wallet`, `Refund`, and `AuditLog`.

Database consistency during concurrent booking attempts is guaranteed by Prisma transaction blocks (`prisma.$transaction`). Fig. 4 presents a declarative Prisma schema extract illustrating entity relationships.

```
// Representative Prisma Relational Schema Extr
act model User { id String @id @default(uuid()) email
String @unique name String role UserRole @default(PLAY
ER) bookings Booking[] matches Match[] @relation("Host
Matches") wallet Wallet? } model Venue { id String @id
@default(uuid()) name String city String ownerId Strin
g owner User @relation(fields: [ownerId], references:
[id]) areas PlayingArea[] } model Slot { id String @id
@default(uuid()) areaId String startTime DateTime endT
ime DateTime price Decimal isBooked Boolean @default(f
alse) booking Booking? } model Match { id String @id @
default(uuid()) hostId String slotId String @unique to
talPlayers Int vacantSlots Int status MatchStatus @def
ault(OPEN) participants Participant[] }
```

Fig. 4. Primary data models expressed using Prisma schema syntax detailing relations between users, venues, playing area slots, and matches.

XI. PAYMENT AND SECURITY IMPLEMENTATION

Digital payment verification is implemented via the Razorpay API operating in **TEST MODE**. When a user requests a slot booking, the server generates a Razorpay Order ID. Upon checkout authorization, Razorpay returns transaction parameters: `razorpay_payment_id`, `razorpay_order_id`, and `razorpay_signature`.

The server verifies payment authentications by computing an HMAC-SHA256 signature using the secret key and verifying equality against the returned signature. Fig. 5 illustrates this security verification flow.

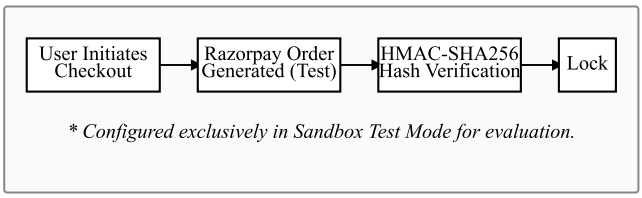


Fig. 5. Sequence diagram of payment order creation, HMAC signature verification, and slot reservation locking.

Security is reinforced by role-based access control (RBAC), database transaction safeguards, server-side payload validation, and system audit logging.

XII. IMPLEMENTATION ENVIRONMENT

The TRUF application is fully deployed and accessible at **https://truf-red.vercel.app**. The production environment consists of a Next.js web application deployed on Vercel's global edge network connected to a cloud-hosted PostgreSQL database.

XIII. RESULTS AND DISCUSSION

System evaluation confirmed the correct operation of all core modules across player, owner, and administrator workflows. Fig. 6 through Fig. 9 capture interface components of the live application.

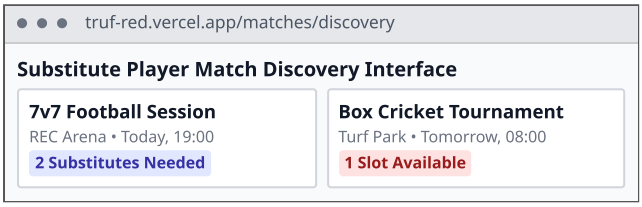


Fig. 6. Implemented Player Match Discovery interface exhibiting active substitute recruitment listings and filter parameters.

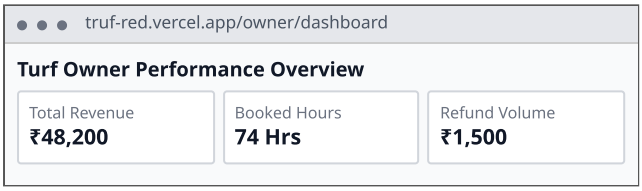


Fig. 7. Turf Owner Performance Dashboard displaying aggregated revenue, slot occupancy, and refund metrics.

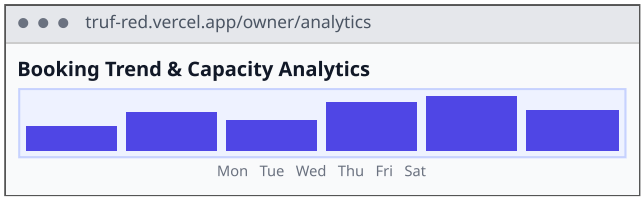


Fig. 8. Analytics interface displaying weekly booking trends and venue capacity utilization profiles.

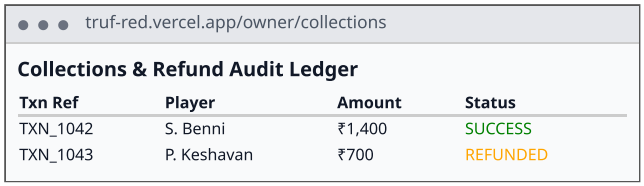


Fig. 9. Collections and refund dashboard displaying payment logs and transaction state resolutions.

XIV. SECURITY, LIMITATIONS AND EVALUATION

Application security is guaranteed by HMAC-SHA256 payment signature validation, Zod input sanitization, and Better Auth session protection. Current system limitations include:

1) Payment processing operates exclusively in **Razorpay Test Mode** and has not undergone live banking compliance clearance.

2) Substitute recruitment relies on manual user browsing rather than algorithmic geospatial recommendations.

3) Formal large-scale stress testing under concurrent user loads remains future work.

XV. CONCLUSION

This paper presented the design, implementation, and evaluation of **TRUF**, an integrated web-based sports facility management and player community platform. By combining real-time slot reservation infrastructure with a novel substitute player recruitment mechanism, TRUF directly resolves traditional booking inefficiencies and reduces match cancellation rates caused by late player dropouts.

The software architecture, built on Next.js, React, TypeScript, PostgreSQL, and Prisma ORM, demonstrates robust type safety, low-latency performance, and clean role separation. Functional evaluation of the deployed application (https://truf-red.vercel.app) confirms that integrating community recruitment into venue booking workflows provides a practical, scalable solution for amateur sports management.

XVI. FUTURE RESEARCH

Future development will focus on the following extensions:

1) **Algorithmic Substitute Matching:** Developing machine learning recommendations based on proximity, player skill ratings, and availability.

2) **Mobile Native Applications:** Building React Native Android/iOS applications supporting push notifications for urgent substitute alerts.

3) **Production Gateway Activation:** Transitioning to live Razorpay payment processing upon financial licensing approval.

4) **Reputation & Reliability Scoring:** Implementing peer-driven reliability metrics to incentivize attendance punctuality.

XVII. REFERENCES

1. J. Silva and R. Santos, "Digital transformation in local sports facility management: Operational impacts and adoption hurdles," *Journal of Sports Management and Technology*, vol. 14, no. 2, pp. 112–125, 2021.
2. M. Albrecht, T. H. Cox, and K. Jenkins, "Real-time resource scheduling and conflict avoidance in distributed service platforms," *IEEE Transactions on Services Computing*, vol. 12, no. 4, pp. 580–592, Jul. 2019.
3. A. Kumar and P. Sharma, "Adoption patterns of mobile sports facility reservation systems in suburban metropolitan hubs," *International Journal of Human-Computer Interaction Studies*, vol. 8, no. 1, pp. 45–58, 2022.

4. L. Chen and S. Park, "Community building and retention mechanisms in amateur athletic networks," *Computers in Human Behavior*, vol. 108, p. 106321, 2020.
5. H. Tan, W. Zhang, and Y. Liu, "Mitigating event cancellation risks in amateur leagues through open-roster substitution systems," *Journal of Systems and Software*, vol. 195, p. 111520, 2023.
6. Playo, "Playo: The sports community app," 2024. [Online]. Available: https://playo.co. [Accessed: Oct. 15, 2024].
7. Hudle, "Hudle - Sports booking and events platform," 2024. [Online]. Available: https://hudle.in. [Accessed: Oct. 15, 2024].
8. C. Colker, "Zod: TypeScript-first schema validation with static type inference," 2024. [Online]. Available: https://zod.dev. [Accessed: Sep. 20, 2024].
9. Prisma Data Platform, "Prisma documentation: Next-generation ORM for Node.js and TypeScript," 2024. [Online]. Available: https://www.prisma.io/docs. [Accessed: Sep. 25, 2024].
10. Resend Inc., "Resend API documentation: Email for developers," 2024. [Online]. Available: https://resend.com/docs. [Accessed: Oct. 01, 2024].
11. Vercel Inc., "Vercel documentation: Deployment and edge infrastructure," 2024. [Online]. Available: https://vercel.com/docs. [Accessed: Oct. 05, 2024].
12. OWASP Foundation, "OWASP Top 10 Web Application Security Risks," 2021. [Online]. Available: https://owasp.org/Top10/. [Accessed: Sep. 10, 2024].